ISO/IEC 60559 Floating-Point Support Annex for C++

Document #:	D4212R1
Date:	2026-08-27
Project:	Programming Language C++
Audience:	SG6

Reply-to:	Guy Davidson <gd@6it.dev>

Contents

1 Colophon

The source of this paper is maintained at the author's paper repository. It is built using Michael Park's proposal authoring framework. Issue submissions and pull requests are welcome.

2 Revision History

R0: Initial proposal with design and wording sketch.

R1: Wording sketch partially replaced with draft wording.

3 Abstract

This paper proposes adding a new normative Annex to [N5054] (Working Draft, Programming Languages — C++) specifying support for floating-point arithmetic as defined by [60559] (ISO/IEC 60559:2020 — Information technology — Microprocessor Systems — Floating-Point arithmetic. ISO. pp. 1–74). The C standard provides a similar annex, but it is not directly suitable for C++ due to differences in language semantics,

constant evaluation, templates, and the standard library. This proposal integrates [60559] semantics into C++ in a manner consistent with existing language rules and implementation practice.

4 Motivation

C++ implementations largely target hardware which supports [60559], yet the Standard does not require consistent semantics for rounding, exceptions, NaN propagation, or reproducibility. This gap leads to portability issues and limits the reliability of numerical software.

Existing facilities such as `<cfenv>` are underspecified and difficult to use correctly in modern C++ contexts, especially with optimization and concurrency.

5 Design Overview

This proposal introduces a new Annex specifying ISO/IEC 60559 conformance. There are nine clauses that need to be considered from [60559], clauses 3 to 11. [60559] offers binary and decimal floating-point arithmetic features. This proposal will only introduce binary floating-point feature support.

This overview iterates through the relevant clauses of [60559] describing what support is offered or needed by [N5054] to implement the features.

5.1 Floating point formats [60559] clause 3

[60559] specifies three binary formats, with encodings of length 32, 64 and 128 bits. [N5054] specifies extended floating point types in [basic.extended.fp], three of which correspond with these types. They are supported via the feature macros `__STDCPP_FLOAT32_T__`, `__STDCPP_FLOAT64_T__` and `__STDCPP_FLOAT128_T__`. This section also includes implementation-defined extended and possibly wider types beyond the standard interchange formats.

{Design question - should these three types be the [60559] types or should we specify that float, double and long double are these types?}

Conversion from floating-point values to integer values is covered in [expr.arith.conv].

Conversion between floating-point types and decimal character sequences is quite underspecified. [lex.fcon] §3 states "If the scaled value is not in the range of representable values for its type, the program is ill-formed. Otherwise, the value of a floating-point-literal is the scaled value if representable, else the larger or smaller representable value nearest the scaled value, chosen in an implementation-defined manner."

5.2 Rounding [60559] clause 4

While describing rounding, this clause also defines the nature of attributes, which are not limited to rounding.

Attribute is sadly an overloaded term, and [60559] defines attributes in a different way to C++. [60559] specifies attributes as something logically associated with a program block to modify its numerical and exception semantics. A user can specify a value for an attribute parameter statically or dynamically. For example, value changing optimisation attributes can be specified on the command line, while rounding attributes can be specified in a function call at runtime. All attempts shall be made to avoid ambiguity.

There are two rounding considerations: rounding to nearest and directed rounding. Five rounding modes are specified in [60559]. In [N5054] at [round.style] four of these modes are specified as part of the enumeration `float_round_style`. They map to:

IEC 60559 identifier	C++ identifier
roundTiesToEven	round_to_nearest
roundTowardsPositive	round_toward_infinity
roundTowardsNegative	round_toward_neg_infinity
roundTowardsZero	round_toward_zero

5.3 Operations [60559] clause 5

The C++ standard library also makes available the facilities of the C standard library, suitably adjusted to ensure static type safety. Support for the operations specified in clause 5 of [60559] is mostly achieved by reference to the C standard library. Some functions are superseded by C++ standard library functions. Some operations are supported by infix operator notation.

The wording needs to reflect the list of operations specified in this clause and the corresponding library functions, with additional paragraphs detailing any clarifications, for example [60559] requires that some of its operations be provided for mixed operand formats, which is covered by the usual arithmetic conversions and the function-call argument conversions.

Consideration needs to be given to the return statement, and how conversion should happen when returning to a different type.

5.4 Infinity, NaNs and sign bit [60559] clause 6

The C standard library provides macros and functions for NaN, signaling NaN and infinity. Consideration needs to be given to how quiet and signaling NaNs are differentiated and denoted.

By default, quiet NaNs are delivered when an invalid operation is executed. Sign bits of NaNs come into play under some operations, for example negation. Specification is required for the sign of zero.

5.5 Exceptions and default exception handling [60559] clause 7

When an operation raises a floating-point exception, default exception handling is assumed: the flag is set, a default result is delivered and execution continues.

5.6 Alternate exception handling attributes [60559] clause 8

Implementations might provide alternate exception handling as an extension, but this annex should not specify any.

5.7 Recommended operations [60559] clause 9

This clause specifies additional operations, such as mathematical functions, floating-point environment controls, reduction operations such as dot product, sum of squares and so on, min and max functions, and payload operations. Many of these are already defined in the C++ standard.

5.8 Expression evaluation [60559] clause 10

When combining operations, the order of execution is significant. Intermediate types need to be considered, as well as possible conversion to the final result format. The same problem is addressed regarding parameters and function values.

There is also discussion of value changing optimisations. This greatly stretches the "as-if" rule since implementations often offer compiler flags which relax restrictions. This feeds into the next clause.

5.9 Reproducibility [60559] clause 11

While steps to reproducibility is specified in this clause, it is not specified in the C++ standard. Efforts are being made in [P3375R3]. This proposal seeks to specify what we have before improving conformance in this way.

5.10 Floating-point environment

The floating-point environment consists of the floating-point exception status flags and the rounding-direction control modes. Implementations may provide additional information as an extension.

During translation, the floating-point environment is not specified and is implementation-defined. C has an `FENV_ACCESS` pragma but [cfenv.syn] explicitly remarks that it is not required. Constant expressions are evaluated as if at execution time.

At startup, floating-point exception status flags are cleared, the dynamic rounding direction mode is rounding to nearest and the dynamic rounding precision mode is set so that results are not shortened. Trapping is disabled on all floating-point exceptions.

Automatic initialisation is done at execution time, or as if at execution time, while static and thread-local object initialisation is done at translation, or as if at translation time.

The environment can be changed at execution time using the functions declared in the `<cfenv>` header.

6 Draft wording

This wording is based on [N5054]. This annex should be located at Annex F to prevent ambiguity between C and C++ about "Annex F".

6.1 Annex F (normative) IEC 60559 floating-point arithmetic [iec559]

6.1.1 F.1 General [iec559.general]

[1] This annex specifies C++ language support for the *IEC 60559* floating-point standard. *The IEC 60559 floating-point standard* is specifically Floating-point arithmetic (ISO/IEC 60559:2020), also designated as *IEEE Standard for Floating-Point Arithmetic* (IEEE 754–2019). IEC 60559 generally refers to the floating-point standard, as in IEC 60559 operation, IEC 60559 format, etc.

[2] The IEC 60559 floating-point standard specifies decimal, as well as binary, floating-point arithmetic. It supersedes *IEEE Standard for Radix-Independent Floating-Point Arithmetic* (ANSI/IEEE 854–1987) which generalized the *binary arithmetic standard* (IEEE 754-1985) to remove dependencies on radix and word length.

7 References

[60559] ISO/IEC JTC 1/SC 25 (May 2020). ISO/IEC 60559:2020 — Information technology — Microprocessor Systems — Floating-Point arithmetic. ISO. pp. 1–74.
https://www.iso.org/standard/80985.html

[N5054] Thomas Köppe. 2026-07-16. Working Draft, Programming Languages — C++.
https://wg21.link/n5054

[P3375R3] Guy Davidson. 2025-05-18. Reproducible floating-point results.
https://wg21.link/p3375r3